```
from google.colab import files

uploaded = files.upload()
```

Choose files No file chosen Cancel upload

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import zipfile
import os

# Unzip the file if it's an archive
zip_file_path = 'archive (3).zip'
if os.path.exists(zip_file_path):
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall('.') # Extract all contents to the current directory
    print(f"Unzipped {zip_file_path}")
else:
    print(f"Warning: {zip_file_path} not found. Ensure the CSV is directly available or correct the zip file name.")

import pandas as pd
df = pd.read_csv("train_u6lujuX_CVtuZ9i.csv")

df.head()
```

```
Unzipped archive (3).zip
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amo
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	

```
df.shape
```

```
(614, 13)
```

```
df.columns
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

```
df.isnull().sum()
```

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64

```
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df['Married'].fillna(df['Married'].mode()[0], inplace=True)
df['Dependents'].fillna(df['Dependents'].mode()[0], inplace=True)
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0], inplace=True)

df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].median(), inplace=True)

df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)
```

```
/tmp/ipykernel_4790/3232930942.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
/tmp/ipykernel_4790/3232930942.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Married'].fillna(df['Married'].mode()[0], inplace=True)
/tmp/ipykernel_4790/3232930942.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Dependents'].fillna(df['Dependents'].mode()[0], inplace=True)
/tmp/ipykernel_4790/3232930942.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Self_Employed'].fillna(df['Self_Employed'].mode()[0], inplace=True)
/tmp/ipykernel_4790/3232930942.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
/tmp/ipykernel_4790/3232930942.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].median(), inplace=True)
/tmp/ipykernel_4790/3232930942.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)
```

```
df.isnull().sum()
```

	0
Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0

dtype: int64

```
df = df.drop('Loan_ID', axis=1)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

```
X = df.drop('Loan_Status', axis=1)

y = df['Loan_Status']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Records:", len(X_train))
print("Testing Records:", len(X_test))
```

```
Training Records: 491
Testing Records: 123
```

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)
model.fit(X_train, y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=42)
```

```
y_pred = model.predict(X_test)

print(y_pred[:10])
```

```
[1 1 1 1 1 1 1 1 1 1]
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy =", round(accuracy*100,2), "%")
```

```
Accuracy = 77.24 %
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

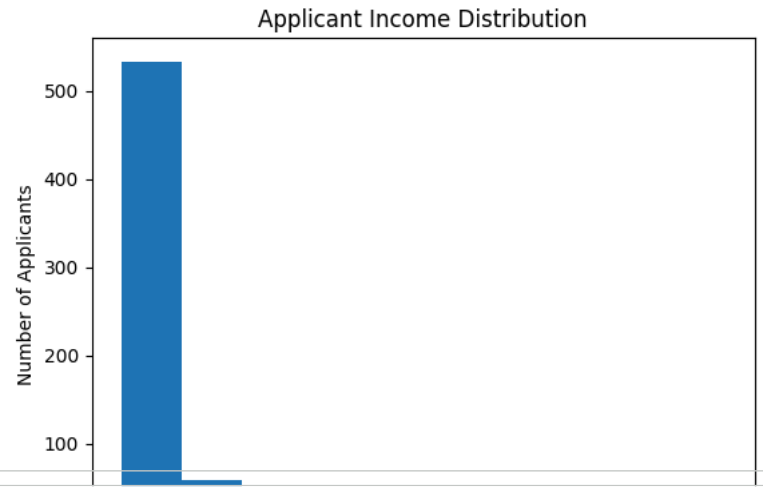
```
[[18 25]
 [ 3 77]]
```

```
import matplotlib.pyplot as plt

plt.hist(df['ApplicantIncome'])

plt.title("Applicant Income Distribution")
plt.xlabel("Income")
plt.ylabel("Number of Applicants")

plt.show()
```




```
import pickle

with open("loan_model.pkl", "wb") as f:
    pickle.dump(model, f)

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
import os

print(os.listdir())
```

```
['.config', 'loan_model.pkl', '12.jpg', 'test_Y3wMUE5_7gLdaTN.csv', 'train_u6lujuX_CVtuZ9i.csv', 'archive (3).zip', 'sample_
```

```
from google.colab import files

files.download("loan model.pkl")
```